# Requirement-aware multi-lift dispatch: restricted waiting and accessibility trade-offs

**Abstract**

Priority service in a lift group shortens waits for passengers who request it and lengthens waits for others, a cost that completed-trip waiting means can hide. This paper represents occupancy, direction compatibility and priority requests as an explicit rule base of dispatch, pickup and boarding rules, so that each passenger requirement can be traced to the rule that acts on it and to the outcome that evaluates it. The rules are evaluated with a restricted mean waiting time over every arriving passenger, reported separately for priority and general requests. In a 16,848-run campaign covering 108 building and demand scenarios, with matched arrival streams and 22,998,183 passenger records, switching rules on one at a time attributes the effects: priority-first boarding alone produces most of the priority benefit and most of the cost to others, while the dispatch bonus and pickup exception add little. With door and transfer delays, the full rule base reduces priority-request waiting from 340.66 to 132.51 model time units relative to an occupancy-matched reference, while general-request waiting rises by 7.26 units (95% paired scenario-bootstrap interval 5.34 to 9.40) and overall completion does not change detectably. Against collective dispatch, general-request waiting falls instead, so the baseline determines the apparent cost. The pattern holds for three dispatch bonuses and horizons from 300 to 1,200 units. A 279,936-run historical campaign shows the evaluation risk: completed-trip means reported a 5.32% overall gain for a configuration whose general-request waiting did not improve. The rule base makes priority an explicit, auditable service-allocation decision.

*Keywords:* lift group control, rule-based decision support, knowledge representation, priority service, restricted mean waiting time, simulation

## 1. Introduction

A lift can arrive promptly and still provide poor service if the car has no room for the passengers waiting. Priority boarding raises a second question: how should a group of cars serve passengers who explicitly request priority, for example because of an accessibility need, while keeping reasonable service for everyone else? A building-wide average can conceal both an unequal allocation of waiting time and passengers who never board during the observation period.

For a building operator, granting priority is a decision about how service is allocated, and it needs two kinds of support. The first is a representation in which each passenger requirement is explicit and can be traced to the controller action that serves it. The second is an evaluation that shows who gains and who pays. Current dispatch research offers limited help with either. Metaheuristic and multi-objective controllers fold requirements into weighted or constrained objectives [1, 2, 3], and learned dispatchers encode them in a reward and a policy network

[4, 5], so the contribution of each requirement to the outcome is hard to isolate. Evaluations often report building-wide or completed-trip averages, which can hide the cost of priority to other passengers.

Occupancy-aware group dispatch, traffic-aware learning and priority scheduling have already been studied [6, 4, 7], so occupancy sensing, coordinated dispatch and accessibility priority are not claimed as new concepts here. The question is narrower: when occupancy and priority requirements are represented as explicit rules, which rule produces which effect, who bears the cost, and do the conclusions survive an evaluation that includes every arriving passenger?

The study answers this question with a requirement-aware rule base and matched-arrival simulation. A 16,848-run campaign on a corrected model, with identical arrival streams across configurations and complete passenger-event records, provides the primary evidence. It switches the priority rules on one at a time, adds door and transfer delays, varies the dispatch bonus and evaluates every arrival over a common horizon. An earlier 279,936-run factorial campaign shows how completed-trip means can misstate the effect of a priority rule when completion differs between configurations.

The paper makes four contributions.

1. *Requirement-to-rule representation.* Passenger and car facts are mapped to three explicit rules for dispatch, pickup and boarding, and each rule is linked to the outcome used to evaluate it (Figure 1, Table 1). Every requirement can therefore be traced from fact to action to outcome.
2. *All-arrival evaluation.* The rules are evaluated with a restricted mean waiting time that includes passengers who have not boarded and is reported separately for priority and general requests. The historical campaign shows how completed-trip means can misstate the conclusion.
3. *Rule-level attribution.* Matched-arrival experiments attribute the effect of priority to individual rules: priority-first boarding alone produces most of the benefit and most of the cost. The trade-off produced by the full rule base holds under door and transfer delays, three dispatch bonuses and four horizons, with paired scenario-bootstrap intervals.
4. *Decision support for a priority policy.* The findings are turned into guidance for specifying and auditing a priority policy (Section 6.2), and all code, data and passenger records are openly archived.

No learning algorithm or physical ride-comfort controller is proposed.

Section 2 reviews related work. Section 3 defines the rule base, Section 4 the experiments and outcomes, and Section 5 reports the results. Section 6 discusses their use for setting a priority policy and their limitations, and Section 7 concludes.

## 2. Related work

### *2.1. Lift group control*

Lift group control assigns hall calls to cars so as to reduce waiting and journey times under changing traffic. Earlier approaches include fuzzy control [8, 9, 10], heuristic search [11], genetic and tabu search [12, 13], multi-agent reinforcement learning [14] and combinatorial optimisation [15], and double-deck installations add further decisions [16]. Recent work has followed several lines. Metaheuristic controllers have been reviewed by Hanif and Mohammad [1];

Vodopija et al. formulate group control as a constrained multi-objective optimisation problem [2]; Beamurgia et al. improve the waiting-time and energy performance of a bi-objective genetic algorithm through passenger-flow estimation [3]; and Wu and Yang study scheduling algorithms under complex traffic patterns [17]. Heuristic dispatch has also been analysed with a finite-state-machine representation of the group [18]. Learning-based methods include occupancy-aware deep learning of usage patterns [6], traffic-pattern-aware deep reinforcement learning [4], and a combination of imitation learning and deep reinforcement learning for a semi-Markov formulation of dispatch [5]. Destination control, in which passengers register their destinations before boarding, raises its own arrival-modelling and planning questions [19, 20].

Priority for passengers with special needs has been addressed by detecting such passengers with image analytics and scheduling them first [7], and camera-assisted group control and passenger detection have been studied more broadly [21, 22, 23]. Occupancy tracking in buildings [24] and thermal-camera assessment of occupant comfort [25] concern distinct measurements and do not justify inferring a passenger's accessibility requirements from appearance. In the present experiments, a priority request is a supplied binary input, not the output of a camera or a diagnosis. The methods above mainly optimise aggregate waiting, journey time or energy; how the cost of priority is distributed across passenger groups is not their primary outcome. The closest systems [6, 4, 7] are not reimplemented here.

### *2.2. Rule-based decisions, priority and fairness*

A rule base is one of the most common and natural explainable frameworks for knowledge representation, which is why rule-based systems remain a key approach in explainable artificial intelligence [26]. In scheduling, explicit priority and dispatching rules are valued for transparency and speed: they have been compared with heuristics for online scheduling [27], evaluated for decentralised multi-project scheduling [28], designed automatically by genetic programming [29] and combined with metaheuristics in interpretable rule-based schedulers [30]. When people are served, priority must also be weighed against fairness, as in rescue-team deployment that considers both [31], equitable distribution of work among delivery couriers [32] and appointment scheduling under a fairness policy [33].

These studies usually build priority or fairness into the objective or into the design of the rules. The present study takes the complementary route of a decision-support evaluation: the rules are fixed and explicit, and the evaluation reports the service each group receives, so that the trade-off is visible rather than resolved inside a composite score. The representation is a small set of facts and rules rather than a learned policy, ontology reasoner or general-purpose knowledge graph. Its purpose is traceability: each requirement maps to an action and to the outcome used to judge it.

### *2.3. Evaluating service over a finite horizon*

Simulation is the usual way to compare dispatch rules; standards recommend round-trip-time calculation for conceptual design before simulation [34, 20]. Simulated performance depends on the demand and movement models, as occupant-presence models [35], uncertain passenger-arrival formulations [19] and independently developed simulators, including an open-source tool [36, 37, 38, 39], make clear. How long individual passengers wait matters, not only the average: waiting times have been examined in relation to stress in human–lift interaction [40].

Completed-trip waiting is conditional on reaching the destination before observation ends, and this condition can differ between controllers. A lower conditional mean can coexist with more unserved passengers, so it cannot alone identify a population-wide service gain. The present study therefore evaluates a restricted mean waiting time from arrival to boarding over all generated passengers, the waiting analogue of the restricted mean survival time used for censored time-to-event outcomes [41]. It reports destination completion separately and distinguishes group means from averages of within-run ratios.

## 3. Requirement representation and dispatch rules

### *3.1. State, requirements and rule structure*

At time $t$, car $c$ is represented by its floor $x_c$, direction $d_c \in \{-1, 0, 1\}$, capacity $C$, onboard passenger set and target floors. Its load ratio is $\rho_c = n_c/C$, where $n_c$ is the number of passengers on board. A request contains an arrival time, origin, destination and priority indicator $p_i \in \{0, 1\}$. The priority indicator represents an externally supplied request for prioritised service. It does not represent a diagnosis, impairment severity or an inferred instability risk.

Figure 1 summarises the rule base. Facts about cars, calling floors and requests feed three rules; each rule drives one controller action; and every action is judged by the same group-level outcomes. Four binary switches turn the rules on and off, which is what allows the experiments to attribute an outcome to a rule. Table 1 lists the same structure by requirement. Capacity is a count of $C = 13$ passengers per car. Neither passenger mass nor the spatial footprint of mobility aids is represented, so an occupancy ratio is a passenger-count ratio and cannot establish that a car has sufficient accessible floor space in a real installation.

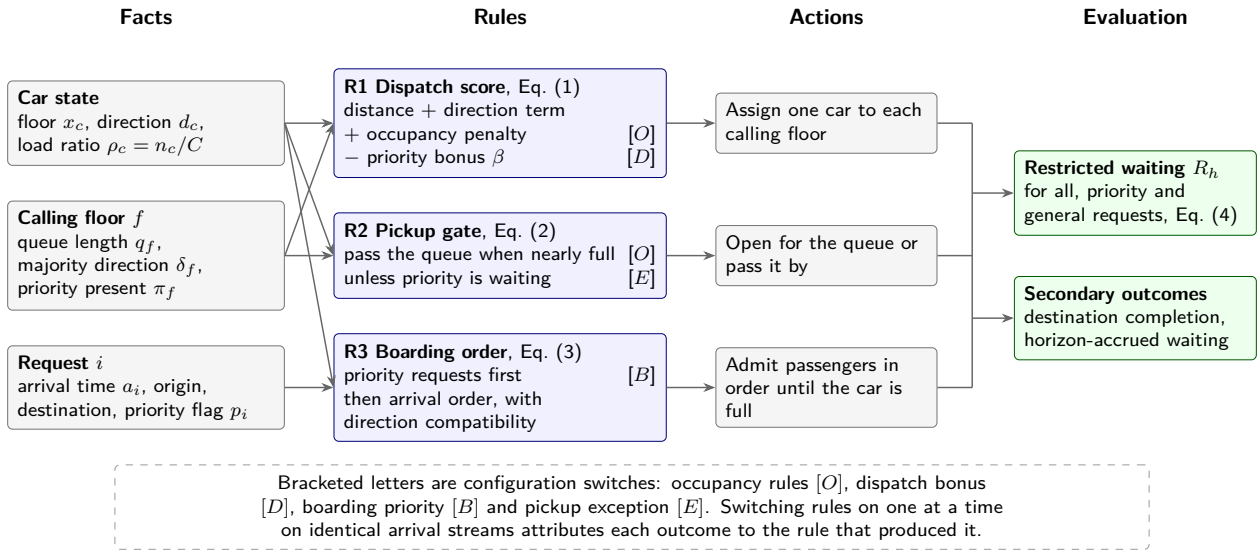



Figure 1: Structure of the rule base. Facts about cars, calling floors and requests feed the dispatch, pickup and boarding rules (Eqs. (1)–(3)); each rule drives one action; and every action is evaluated by restricted waiting for each request group (Eq. (4)) and by the secondary outcomes. The switches $O$, $D$, $B$ and $E$ enable the rules independently.

Table 1: Requirement facts, rule actions and evaluation outcomes.

| Requirement | Facts used | Rule action | Evaluation outcome |
|---|---|---|---|
| Reach a waiting request | Car floor; request floor | Minimise distance, with a direction term (Eq. (1)) | Restricted wait |
| Account for occupied capacity | Passenger count; capacity | Load penalty (Eq. (1)); pickup gate (Eq. (2)) | Boarding and completion |
| Prioritise a supplied request | Priority flag; remaining capacity | Dispatch bonus (Eq. (1)), pickup exception (Eq. (2)) and priority-first boarding (Eq. (3)) | Priority-group restricted wait |
| Preserve service for other requests | Arrival order; direction | Arrival-order boarding within groups, with direction compatibility (Eq. (3)) | General-group restricted wait |

### *3.2. Assignment and pickup*

Every eight time steps, each floor $f$ with a non-empty queue $Q_f$ is assigned a car. Let $\delta_f$ be the majority direction of the queue and $\pi_f = 1$ if any request in $Q_f$ has $p_i = 1$. Four binary switches describe a configuration: the occupancy rules $O$, the priority dispatch bonus $D$, priority-first boarding $B$ and the priority pickup exception $E$. The assigned car minimises

$$\begin{aligned} s_c(f,t) = |x_c - f| + 2.5\,\mathbf{1}[d_c \neq 0,\ \delta_f \neq 0,\ d_c \neq \delta_f] + 10\,O\,\max(0, \rho_c - 0.78) \\ - \beta\,D\,\pi_f\,(1-\rho_c) + \varepsilon(f,t,c), \end{aligned} \tag{1}$$

where $\beta = 2.8$ unless stated otherwise and $\varepsilon \in [0, 0.02)$ breaks ties. The collective reference is the configuration with all four switches off.

When a car reaches an assigned floor, the pickup gate decides whether it opens for the waiting queue. With $n'_c$ passengers remaining after alighting, $\rho'_c = n'_c/C$ and queue length $q_f$, occupancy configurations ($O = 1$) apply, in order,

$$G_c(f) = \begin{cases} 0 & \text{if } \rho'_c \geq 0.92 \text{ and not } \Big(E\,\pi_f = 1 \text{ and } n'_c < C\Big), \\ E\,\pi_f & \text{otherwise, if } \rho'_c \geq 0.82 \text{ and } q_f > \max(1,\, C - n'_c), \\ 1 & \text{otherwise,} \end{cases} \tag{2}$$

and $G_c(f) = 1$ when $O = 0$. The thresholds are fixed and were not tuned using the reported outcomes.

The distance-only nearest-car reference selects the car with minimum $|x_c - f|$, breaking equal distances by car identifier. It has no direction, occupancy or priority term and boards in arrival order. It is an explicitly specified conventional reference rule, not a reproduction of a complete published nearest-car implementation, and it differs from the historical configuration named nearest_car, which uses the same decision logic as the historical collective configuration.

### *3.3. Boarding, movement and delays*

Boarding orders the queue lexicographically by

$$k_i = \Big(B\,(1 - p_i),\ a_i,\ i\Big), \tag{3}$$

so that, when $B = 1$, priority requests board first and each group boards in arrival order $a_i$. Passengers are admitted in this order until the car is full. When the car still carries passengers after alighting, a passenger whose direction differs from the car's travel direction (or, for an idle car, from the queue's majority direction) is skipped.

Cars move one floor per model time step towards their targets, continuing while targets remain ahead and reversing otherwise. A stationary car heads for its nearest target. Equidistant targets are resolved by the iteration order of the target set, a detail that matters for exact replay (Appendix C).

Without delays, transfers occur at the stop and the car moves in the same step. In the delay configurations, a stop incurs five fixed model time units, then one unit for each alighting passenger and then one for each boarding passenger. Transfers are recorded at completion, physical occupancy changes at each transfer and the car cannot move during service. Boarding reserves a cohort using the capacity left after planned alighting, and later arrivals wait for another service cycle. No busy-time term is added to Eq. (1). The delay values are simple assumptions, not calibrated operational measurements.

These abstractions support a comparison of rules within the model; they are not calibrated predictions for a building. No speed adaptation, acceleration, physical jerk or measured energy model is evaluated. The synthetic stability, comfort and guidance accounting of the historical simulator is not part of the corrected model and is not used as an outcome.

## 4. Experimental design and outcomes

### *4.1. Matched-arrival campaign*

The primary evidence comes from a bounded campaign on a corrected model version (Table 2). The diagonal floor/car pairs (8, 2), (16, 4) and (32, 8) are crossed with three population scales, four traffic patterns and three priority-request shares, giving 108 structural scenarios, each run with twelve seeds. Thirteen configurations share identical arrival records within each scenario–seed pair. This was verified with a hash of passenger identifier, origin, destination, arrival time and synthetic attributes.

Nine configurations have no door or transfer delay. Five of them form the rule ablation: an occupancy reference with $O = 1$ and no priority rule, the dispatch bonus alone ($D = 1$), priority-first boarding alone ($B = 1$), the pickup exception alone ($E = 1$), and the full priority rule base ($D = B = E = 1$). Holding the occupancy penalty and pickup gate fixed in all five arms makes the contribution of each priority rule, including the exception, interpretable. Two further full-rule arms use $\beta = 1.4$ and $\beta = 4.2$, and collective dispatch ($O = 0$) and the distance-only nearest car complete the set. Four configurations repeat the occupancy reference, full priority, collective and distance-only arms with the delay model of Section 3.3.

The campaign uses a corrected version of the simulator from the earlier campaigns (Section 4.4). Two corrections apply to every configuration. First, a served or empty target floor is removed when a car stops; the legacy simulator could retain such stale targets and travel to them unnecessarily. Second, the tie-break term $\varepsilon$ in Eq. (1) is a counter-based pseudorandom value keyed by seed, time, calling floor and car, so every configuration sees the same values; the legacy simulator drew it from a configuration-specific random stream shared with unused comfort accounting. Contrasts are therefore computed only within the corrected model. An earlier 5,184-run event-recording campaign on the legacy simulator is superseded by this campaign and reported in Appendix C.

Table 2: Design of the matched-arrival campaign.

| Design element | Value |
|---|---|
| Structural scenarios | Floor/car pairs (8, 2), (16, 4), (32, 8) × population 200, 800, 2,000 × four traffic patterns × priority share 0.02, 0.08, 0.15 = 108 |
| Fixed factors | Carrying share 0.20 and synthetic noise 0.08; neither enters the corrected model's decisions |
| Configurations without delay | Occupancy reference; dispatch only; boarding only; pickup exception only; full priority ($\beta = 2.8$); full priority with $\beta = 1.4$ and $\beta = 4.2$; collective; distance-only nearest car |
| Configurations with delay | Occupancy reference, full priority, collective and distance-only nearest car, each with five fixed units per stop and one unit per transfer |
| Replications | Twelve seeds; identical arrival streams across all thirteen configurations |
| Horizons | Arrivals 0–3,599; observation to $T = 4{,}500$; restricted horizon 900 (sensitivity 300–1,200) |
| Runs and records | 16,848 runs, of which 5,184 with delays; 22,998,183 passenger records |
| Execution | Python 3.12.1 and NumPy 2.3.5; source files and protocol fixed and hashed before execution |

The grid, configurations and analysis were fixed in a protocol before execution. Six invariant checks and a 26-case development smoke test preceded the run, and the stopping rule permitted only recovery of technically missing cases. No case was excluded or repeated on the basis of performance. Every run's passenger events were reconstructed and checked against its summary, including cohort hashes, group counts, transfer order and restricted waiting.

*4.2. Restricted mean waiting time*

For passenger $i$ with arrival time $a_i$ and boarding time $b_i$, observed until $T = 4{,}500$, the waiting time and its restricted mean over a set of passengers $P$ are

$$W_i = \begin{cases} b_i - a_i & \text{if } i \text{ boards before } T, \\ T - a_i & \text{otherwise,} \end{cases} \qquad R_h(P) = \frac{1}{|P|} \sum_{i \in P} \min(W_i, h). \tag{4}$$

Every arrival occurs by time 3,599, so $T - a_i \geq 901$ and $R_h$ is fully observed for every passenger whenever $h \leq 900$. The primary outcome is $R_{900}$, computed for all, priority and general requests within each run and then averaged with equal weight across runs. A boarded passenger counts as served even if alighting is not complete.

Destination completion and horizon-accrued waiting, the uncapped mean of $W_i$, are secondary outcomes. Horizon-accrued waiting is a finite-horizon service burden, not an estimate of eventual waiting beyond observation. The horizon sensitivity recomputes $R_h$ from the retained passenger events at $h = 300$, 600 and 900, all fully observed, and at $h = 1{,}200$ over the 93.81% of arrivals with at least 1,200 units of follow-up.

*4.3. Paired uncertainty*

Intervals resample the 108 structural scenarios with all twelve seeds and all configurations kept together, so every contrast is paired. They use 2,000 bootstrap draws with seed 20261008

and percentile 95% intervals, without multiplicity correction. The intervals describe variation over the fixed design-scenario composition. The scenarios are not a random sample of real buildings, and the intervals do not quantify model calibration uncertainty. Run-level means receive equal weight, and completion is an average of run-level fractions. Each group's wait is reported directly rather than through a single fairness ratio, because ratios of small waits are unstable (Appendix A).

### *4.4. Historical factorial campaign*

An earlier factorial campaign on the legacy simulator provides a cautionary comparison. Seven factors (Appendix A, Table A.1) give 2,916 combinations, which with eight configurations and twelve seeds give 279,936 runs. Each run generates Poisson arrivals with traffic-dependent intensity for 3,600 steps and allows up to 900 clearance steps, with no door or transfer delay. The outputs contain run-level means and counts, not individual waiting or censoring times, so a passenger-level restricted mean cannot be recovered. The campaign is therefore used only to show how completed-trip means behave (Section 5.5). Its intervals use the same bootstrap settings but resample 972 structural blocks defined by floors, cars, population, traffic, priority share and carrying share, keeping all three noise levels and twelve seeds together so that invariant noise replicates are not treated as independent evidence; completion fractions are passenger-weighted. The full experiment history is listed in Appendix B.

## 5. Results

### *5.1. Rule-level attribution*

Table 3 reports the matched-arrival campaign without delays. Priority-first boarding reproduced most of the full rule base's priority benefit (Figure 2). Relative to the occupancy reference, boarding alone reduced priority restricted waiting by 165.13 units ($[-209.61, -124.04]$) while increasing general-request waiting by 6.26 units ($[4.56, 8.20]$). The corresponding full-rule differences were $-165.55$ ($[-209.95, -124.46]$) and $+6.08$ ($[4.27, 8.03]$). Dispatch alone changed priority waiting by $-0.02$ units ($[-0.51, 0.47]$), and the pickup exception alone by $-0.24$ ($[-0.48, -0.06]$). Boarding alone is therefore sufficient to produce most of the effect in this design; the singleton arms do not show that it is necessary or resolve every interaction among rules.

Table 3: Outcomes without delays. Waiting is restricted to 900 model time units and averaged over all 1,296 runs per configuration.

| Configuration | All | Priority | General | Completion (%) |
|---|---|---|---|---|
| Occupancy reference | 203.96 | 200.78 | 204.24 | 88.04 |
| Dispatch only | 203.90 | 200.76 | 204.19 | 88.04 |
| Boarding only | 196.81 | 35.65 | 210.50 | 88.06 |
| Pickup exception only | 203.80 | 200.54 | 204.09 | 88.04 |
| Full priority | 196.64 | 35.22 | 210.33 | 88.05 |
| Collective | 205.38 | 202.09 | 205.67 | 88.02 |
| Distance-only nearest car | 226.12 | 222.42 | 226.41 | 87.09 |

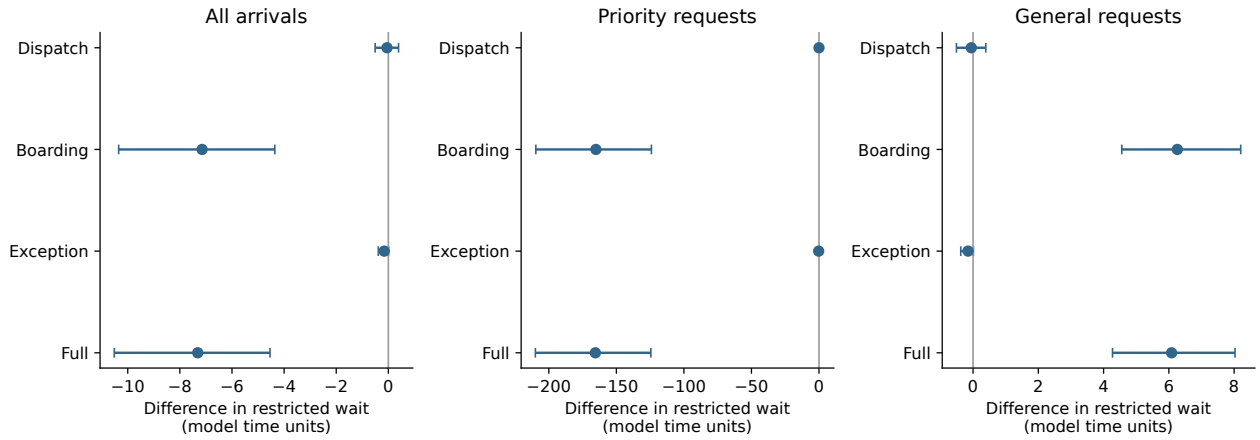



Figure 2: Rule effects relative to the occupancy-matched reference, without delays. Points show paired differences in 900-unit restricted waiting; horizontal bars show 95% scenario-bootstrap intervals. Negative values indicate lower waiting, and each panel uses its own scale. Boarding alone yields most of the priority benefit and of the general-request cost; singleton arms do not constitute a complete factorial interaction analysis.

The full rule base reduced overall restricted waiting by 7.31 units relative to the occupancy reference ($[-10.52, -4.54]$), but changed horizon-accrued waiting by only $+0.36$ ($[-0.53, 1.27]$) and overall completion by $+0.016$ percentage points ($[-0.009, 0.043]$). Priority completion rose by 11.72 percentage points and general completion fell by 1.10 points. A lower capped mean therefore does not imply better eventual service or higher total throughput. Against collective dispatch, overall restricted waiting fell by 8.73 units ($[-12.01, -5.73]$), while general-request waiting rose by 4.66 ($[1.95, 7.27]$).

### *5.2. Door and transfer delays*

Introducing door and transfer delays increased absolute waiting markedly: collective restricted waiting rose from 205.38 to 357.14 units. The 5,184 runs with delays retained the direction of the priority effect (Table 4, Figure 3). Relative to the occupancy reference, full priority reduced priority waiting from 340.66 to 132.51 units, a difference of $-208.15$ ($[-247.76, -170.90]$), or 61.10%. General waiting rose from 344.38 to 351.64, a difference of $+7.26$ ($[5.34, 9.40]$). Overall restricted waiting fell by 8.68 units ($[-11.50, -5.93]$), while the horizon-accrued difference was $-0.84$ ($[-2.51, 0.78]$). General completion fell by 1.55 percentage points ($[-2.07, -1.07]$), and the interval for overall completion included zero.

Table 4: Outcomes with five fixed units per stop and one unit per passenger transfer; 1,296 runs per configuration.

| Configuration | All | Priority | General | Completion (%) |
|---|---|---|---|---|
| Occupancy reference | 344.08 | 340.66 | 344.38 | 79.60 |
| Full priority | 335.40 | 132.51 | 351.64 | 79.63 |
| Collective | 357.14 | 353.40 | 357.48 | 78.30 |
| Distance-only nearest car | 363.57 | 359.42 | 363.94 | 78.10 |

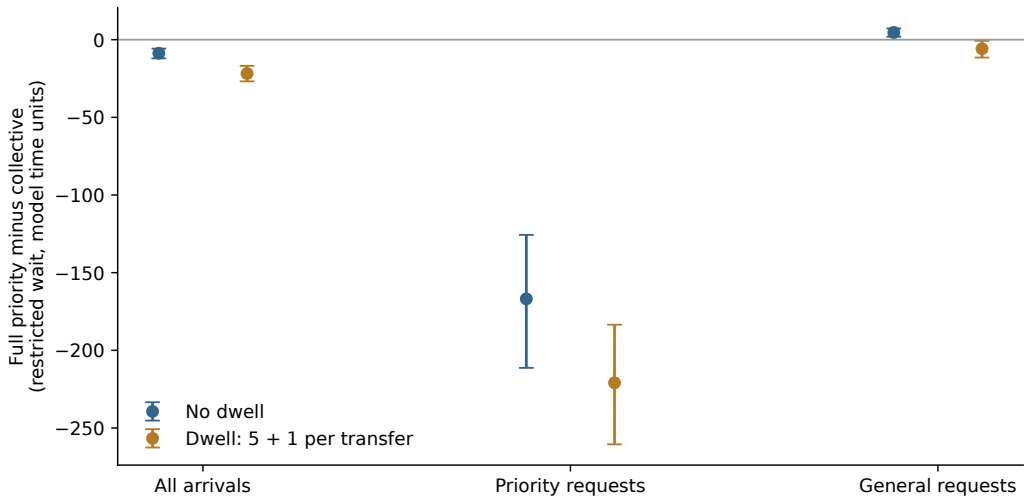



Figure 3: Full priority versus collective dispatch without delays and with five fixed units per stop plus one per transfer. Points are paired differences in 900-unit restricted waiting and bars are 95% scenario-bootstrap intervals, for all, priority and general arrivals. With delays the general-request comparison against collective dispatch changes sign, unlike the general-request cost relative to the occupancy reference (Table 4).

The choice of baseline determines the apparent cost. With delays, full priority also reduced general waiting relative to collective dispatch, by 5.84 units ($[-11.49, -0.74]$), and reduced overall waiting by 21.74 ($[-26.81, -16.82]$). Overall completion rose by 1.33 percentage points ($[0.64, 2.14]$), whereas the general completion difference of $-0.25$ points had an interval spanning zero. These combined comparisons do not isolate priority: under delays, the occupancy rules alone reduce general waiting by about 13 units relative to collective dispatch (344.38 against 357.48 in Table 4), which more than offsets the cost of priority. The occupancy-matched comparison isolates the general-request cost.

### *5.3. Priority bonus sensitivity*

All three bonuses produced closely similar restricted waiting (Table 5, Figure 4). Compared with $\beta = 2.8$, using $\beta = 1.4$ changed priority waiting by $+0.05$ units ($[-0.11, 0.24]$) and using $\beta = 4.2$ changed it by $-0.11$ ($[-0.34, 0.12]$). The corresponding general-request changes were $+0.09$ ($[-0.01, 0.22]$) and $-0.03$ ($[-0.14, 0.09]$). When priority-first boarding is enabled, the outcome is insensitive to the dispatch bonus within this range. This is bounded evidence, not proof of equivalence outside the tested range, and no bonus was selected after inspecting outcomes.

Table 5: Bonus sensitivity with all three priority rules enabled and no delays.

| Configuration | All | Priority | General | Completion (%) |
|---|---|---|---|---|
| Full priority, $\beta = 1.4$ | 196.72 | 35.28 | 210.42 | 88.05 |
| Full priority, $\beta = 2.8$ | 196.64 | 35.22 | 210.33 | 88.05 |
| Full priority, $\beta = 4.2$ | 196.61 | 35.12 | 210.30 | 88.05 |

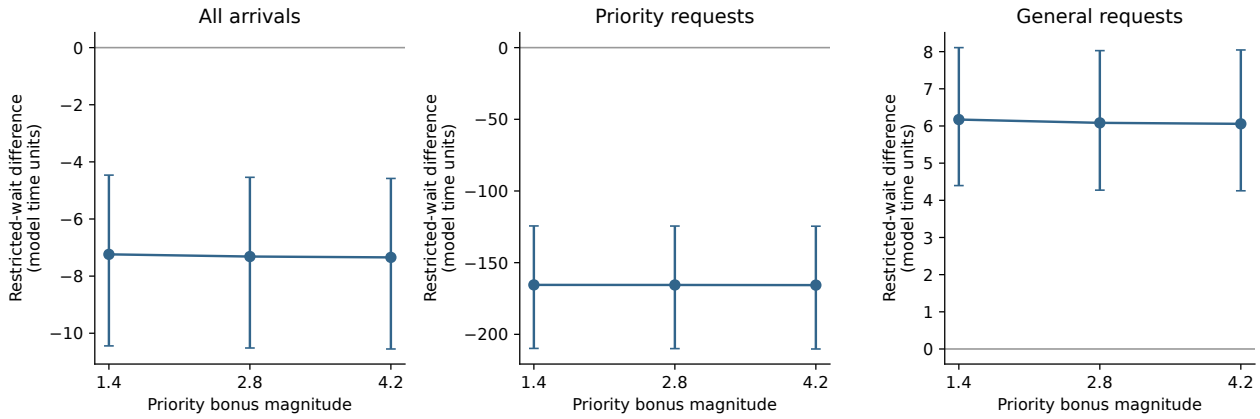



Figure 4: Restricted-wait differences relative to the occupancy reference at bonus magnitudes $\beta = 1.4$, 2.8 and 4.2, with all priority rules enabled and no delays. Bars show 95% scenario-bootstrap intervals. Direct paired intervals for the change between bonus values are reported in Section 5.3.

### *5.4. Restricted-horizon sensitivity*

Table 6 recomputes the occupancy-matched contrasts at four horizons from the retained passenger events; the 900-unit values reproduce Sections 5.1 and 5.2 exactly. At every horizon, with and without delays, full priority lowers priority-request waiting, raises general-request waiting and lowers the capped overall mean, and no interval includes zero. Both the benefit and the cost grow with the horizon, because a longer cap counts more of the longest waits. Against collective dispatch without delays, the general-request difference is small at short horizons and its interval includes zero at 300 and 600 units (0.41, $[-1.01, 1.56]$; 2.12, $[-0.07, 3.98]$). With delays it remains negative except at 1,200 units, where the interval includes zero ($-4.74$, $[-11.71, 1.53]$). The service-allocation conclusion therefore does not depend on the choice of a 900-unit horizon.

Table 6: Full priority minus the occupancy reference in restricted waiting at four horizons; positive values indicate longer waits under full priority. Entries are paired differences with 95% scenario-bootstrap intervals.

| Horizon | Group | Without delays | With delays |
|---|---|---|---|
| 300 | All arrivals | −3.48 [−4.77, −2.27] | −3.37 [−4.49, −2.31] |
| | Priority requests | −60.80 [−75.56, −47.24] | −65.21 [−78.25, −53.04] |
| | General requests | 1.25 [0.73, 1.79] | 1.48 [1.02, 1.95] |
| 600 | All arrivals | −5.72 [−8.09, −3.60] | −6.66 [−8.69, −4.72] |
| | Priority requests | −114.78 [−143.81, −87.59] | −138.94 [−166.17, −114.06] |
| | General requests | 3.42 [2.34, 4.56] | 3.84 [2.77, 5.00] |
| 900 | All arrivals | −7.31 [−10.52, −4.54] | −8.68 [−11.50, −5.93] |
| | Priority requests | −165.55 [−209.95, −124.46] | −208.15 [−247.76, −170.90] |
| | General requests | 6.08 [4.27, 8.03] | 7.26 [5.34, 9.40] |
| 1,200[a] | All arrivals | −7.85 [−11.48, −4.74] | −9.49 [−12.77, −6.25] |
| | Priority requests | −212.68 [−271.06, −159.29] | −270.92 [−322.35, −221.27] |
| | General requests | 9.67 [6.78, 12.85] | 11.49 [8.77, 14.43] |

[a]Computed over arrivals with at least 1,200 units of follow-up (93.81% of arrivals); the other horizons include every arrival.

*5.5. What completed-trip means hide*

The historical campaign shows how the usual evaluation can misstate the same kind of trade-off. Table 7 presents its principal configurations. The historical layered configuration, which added a stability-related dispatch penalty and synthetic exposure accounting to the priority rule, has a conditional mean waiting time of 349.87 units against 369.54 for the collective reference, a reduction of 5.32% (clustered interval 4.10% to 6.56%). The apparent overall gain is carried by priority requests. The layered configuration's general-request wait is 372.40 against 370.81, a paired difference of 1.59 units ($[-8.16, 10.00]$), and its passenger-weighted completion is lower, at 74.45% against 75.97% (Figure 5).

Table 7: Historical completed-trip-only waits and passenger-weighted completion.

| Configuration | Conditional wait | Priority wait | General wait | Completion (%) |
|---|---|---|---|---|
| Priority rule | 350.44 | 129.41 | 373.66 | 74.61 |
| Collective reference | 369.54 | 353.46 | 370.81 | 75.97 |
| Historical layered | 349.87 | 133.19 | 372.40 | 74.45 |
| Occupancy rule | 377.12 | 358.40 | 378.49 | 74.84 |

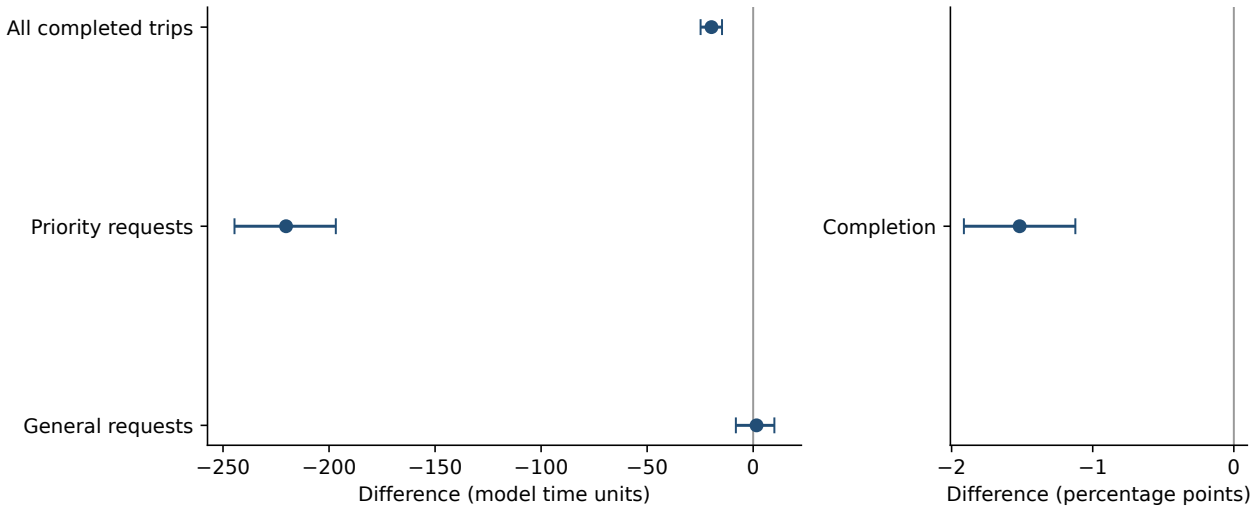



Figure 5: Historical layered configuration versus the collective reference. Points are paired differences in completed-trip group means and in passenger-weighted completion; bars are 95% structural-scenario bootstrap intervals. These conditional differences do not establish an improvement over all arrivals.

Because completion differs, the conditional means describe different passenger populations, and the run-level summaries bound the all-arrival waiting burden only broadly: the bounds for the two configurations overlap (Appendix A). As a sensitivity calculation rather than recovered data, assigning every unresolved passenger 1,000 units gives 445.89 units for the layered configuration and 444.21 for the reference, reversing the ordering of passenger-weighted conditional waiting. This is a different aggregation from the 5.32% run-mean comparison and is not a corrected version of that percentage. A completed-trip mean can therefore report an overall gain for a configuration that does not improve service for general requests, which is why the matched-arrival campaign uses the restricted mean over all arrivals.

## 6. Discussion

### *6.1. A service-allocation trade-off*

In the corrected model, the priority rules produce a large improvement for requests designated for priority service and a smaller, consistent cost for general requests when the occupancy rules are held fixed: 6.08 units without delays and 7.26 units with them. The capped overall mean improves, but horizon-accrued waiting and overall completion show no detectable change, so the rules redistribute service rather than adding capacity. The ablation locates the mechanism in boarding order; within the tested design, the dispatch bonus and the pickup exception contribute little.

The apparent cost depends on the comparison. Against collective dispatch, the general-request difference changes sign once delays are modelled, because the occupancy rules then shorten service for everyone by more than priority costs general requests. A comparison against a weaker reference can therefore hide the cost of priority. The historical campaign shows a second way the cost can be hidden: completed-trip means reported a 5.32% overall gain for a combined configuration whose general-request wait did not improve and whose completion was lower.

Priority may be justified by an explicit service requirement even when it does not minimise the building-wide mean. A controller should then expose that requirement and its cost rather than hide the cost inside a composite score or a conditional average. Whether a given cost is acceptable depends on the building's agreed service policy and passenger requirements; the simulation quantifies the trade-off but does not decide that policy.

### *6.2. Using the rule base to set a priority policy*

Because each requirement is acted on by a named rule and judged by a named outcome, the results can be read as answers to the decisions a building operator faces when introducing priority service (Table 8). Who receives priority is a policy input, represented here by a supplied request flag; it should come from a user-controlled request for assistance rather than from inference. Which mechanism to use is answered by the attribution: in the tested design, boarding order carries most of the effect, so priority can be specified and audited at the boarding rule rather than through dispatch weights, and within the tested range the bonus magnitude needs no tuning. How much cost to accept is a matter of policy, but it should be measured against an occupancy-matched reference, because comparison with collective dispatch can show a gain for general requests that comes from the occupancy rules rather than from priority. Finally, completion and horizon-accrued waiting by group show whether the rules move service between groups or lose capacity.

A real implementation would also need a measured capacity representation that accounts for mobility aids, calibrated movement and dwell times, and a user-controlled way of requesting appropriate assistance. These belong to the facts layer of Figure 1 and can be added without changing how the rules are evaluated.

### *6.3. Explicit rules and learned dispatchers*

The evaluation does not depend on the rules being hand-written. A learned dispatcher [4, 5] or a metaheuristic controller [1, 2] could be audited in the same way, by reporting restricted waiting for each request group against an occupancy-matched reference and by switching

Table 8: Priority-policy decisions supported by the rule base and the evidence from this study.

| Decision | Rule element | Evidence in this study | What to monitor |
|---|---|---|---|
| Who receives priority | Request flag $p_i$ | A policy input; priority shares of 2–15% were tested | Priority share; $R_h$ for each group |
| Which mechanism | Boarding [$B$], bonus [$D$], exception [$E$] | Boarding alone gives most of the benefit and cost; $D$ and $E$ add little (Section 5.1) | $R_h$ by group with each rule switched off |
| How strong | Bonus $\beta$ | Outcomes are insensitive to $\beta$ between 1.4 and 4.2 when boarding priority is on (Section 5.3) | No tuning needed within the tested range |
| What cost is acceptable | General-request $R_h$ | +6.08 units without delays and +7.26 with delays against the occupancy-matched reference (Sections 5.1–5.2) | General-request $R_h$ against an occupancy-matched reference, not collective dispatch |
| Whether capacity is lost | Completion; horizon-accrued waiting | No detectable overall change; general completion falls by 1.10 points without delays and 1.55 with them | Completion and accrued waiting by group |

priority mechanisms on and off where the controller allows it. What explicit rules add is attribution: because each requirement is acted on by a named rule, the ablation can say which rule produced the benefit and which produced the cost. For a learned policy, the same question requires a counterfactual analysis of the policy itself. Explicit rules and learned controllers are therefore complementary: the former can define the requirements and audit outcomes that the latter are expected to meet.

### *6.4. Limitations and generality*

The rule set is fixed and hand-specified. It is not a learned policy or a general knowledge-reasoning formalism, and because the closest published methods [6, 4, 7] were not reimplemented, no claim of superiority over them is made. The one-floor-per-step movement model limits transfer to real installations, and the fixed-plus-transfer delay model is uncalibrated: it tests robustness to one simple assumption, not the full range of real door-service times. The five-arm ablation omits paired-rule arms, and bonus sensitivity was tested only without delays, so interactions among rules and between delay and bonus are not fully identified.

The scenarios cover diagonal floor/car combinations rather than the full historical grid, with fixed carrying and noise levels. Population controls arrival intensity and is not a closed census of occupants. A binary priority indicator omits individual preferences and the space required by mobility aids, and no claim is made that wheelchair occupancy, disability or a camera-derived attribute predicts passenger instability. Physical sensing, motion control and visual or spoken interaction are outside the evaluated mechanism. The restricted horizon is a reporting choice rather than a claim about eventual service, so completion and horizon-accrued waiting are reported alongside it. The bootstrap describes variation over the tested scenario composition, not uncertainty about deployment performance.

Exact replay depends on the Python version, because equidistant movement targets are resolved by set iteration order (Appendix C). The historical archive reproduces under Python

3.9 and the corrected campaign under Python 3.11 or later, so bit-exact verification requires the matching interpreter. The sensitivity of the conclusions to a different tie rule was not tested.

## 7. Conclusions

Representing passenger requirements as explicit dispatch, pickup and boarding rules makes it possible to say which rule changes how a lift group allocates service, and to whom. In a 16,848-run campaign with matched arrivals, priority-first boarding alone generates most of the reduction in priority-request waiting and most of the added waiting for general requests. With door and transfer delays, the full rule base reduces priority-request restricted waiting by 208.15 units (61.10%) relative to an occupancy-matched reference and increases general-request waiting by 7.26 units. The pattern holds at horizons from 300 to 1,200 units and across three dispatch bonuses. The capped overall mean falls, but horizon-accrued waiting and overall completion do not change detectably, so the rules reallocate service rather than adding capacity. Against collective dispatch, general-request waiting falls instead, which shows that the chosen baseline determines the apparent cost.

The evaluation result is that completed-trip means can misstate such trade-offs. In the historical campaign they reported an overall gain for a configuration with unchanged general-request waiting and lower completion. A restricted mean over all arrivals, reported by request group alongside completion and against an occupancy-matched reference, shows who gains and who bears the cost. Together, the rule base and this evaluation turn priority service into an explicit decision that a building operator can specify, audit and revise.

## 8. Data and code availability

All code, data and reproduction instructions are archived in a public repository with a persistent identifier [42]; the link is withheld for anonymised review and is given on the title page. The archive contains the 279,936-row historical CSV (SHA-256 7654ba08…e740d) with its post-run code snapshot; the legacy event-recording campaign of Appendix C; and, for the corrected campaign, the frozen protocol, executed source revision, per-run outcomes, paired intervals, horizon sensitivity and a checksum-verified archive of all 22,998,183 passenger records. REPRODUCING.md gives the restoration, verification and replay commands, including the Python versions required for exact replay. Code is released under the MIT licence and data under the Creative Commons Attribution 4.0 licence. All records are synthetic simulation outputs; no human-participant data were collected.

## 9. Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## 10. Declaration of competing interests

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## 11. Declaration of generative AI and AI-assisted technologies in manuscript preparation

During the preparation of this work, the author used ChatGPT (OpenAI) solely to improve the grammar, language, clarity and readability of the manuscript. The tool was not used for research design, data generation, simulation, implementation, data analysis, interpretation of results, or formulation of scientific conclusions. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

## References


[1] M. Hanif, N. Mohammad, Metaheuristic algorithms for elevator group control system: a holistic review, Soft Computing 27 (2023) 15905–15936. doi:10.1007/s00500-023-08843-0.

[2] A. Vodopija, J. Stork, T. Bartz-Beielstein, B. Filipič, Elevator group control as a constrained multiobjective optimization problem, Applied Soft Computing 115 (2022) 108277. doi:10.1016/j.asoc.2021.108277.

[3] M. Beamurgia, R. Basagoiti, I. Rodríguez, V. Rodríguez, Improving waiting time and energy consumption performance of a bi-objective genetic algorithm embedded in an elevator group control system through passenger flow estimation, Soft Computing 26 (2022) 13673–13692. doi:10.1007/s00500-022-07358-4.

[4] J. Wan, K. Lee, H. Shin, Traffic pattern-aware elevator dispatching via deep reinforcement learning, Advanced Engineering Informatics 61 (2024) 102497. doi:10.1016/j.aei.2024.102497.

[5] J. Wan, K. Lee, H. Shin, A Hybrid Approach of Imitation Learning and Deep Reinforcement Learning with Direct-Effect Update Interval for Elevator Dispatching, ACM Transactions on Cyber-Physical Systems 10 (2026) 1–19. doi:10.1145/3790252.

[6] S. Wang, X. Gong, M. Song, C. Y. Fei, S. Quaadgras, J. Peng, P. Zou, J. Chen, W. Zhang, R. J. Jiao, Smart dispatching and optimal elevator group control through real-time occupancy-aware deep learning of usage patterns, Advanced Engineering Informatics 48 (2021) 101286. doi:10.1016/j.aei.2021.101286.

[7] W. Han, A. Y. Chen, N.-W. Chi, S.-H. Hsieh, Elevator priority scheduling with deep learning based image analytics for people with special needs, Advanced Engineering Informatics 62 (2024) 102794. doi:10.1016/j.aei.2024.102794.

[8] C. B. Kim, K. Seong, H. Lee-Kwang, J. Kim, Y. B. Lim, A fuzzy approach to elevator group control system, IEEE Transactions on Systems, Man, and Cybernetics 25 (1995) 985–990. doi:10.1109/21.384263.

[9] C. Kim, K. Seong, H. Lee-Kwang, J. Kim, Design and implementation of a fuzzy elevator group control system, IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans 28 (1998) 277–287. doi:10.1109/3468.668960.

[10] J. Fernandez, P. Cortes, J. Munuzuri, J. Guadix, Dynamic Fuzzy Logic Elevator Group Control System With Relative Waiting Time Consideration, IEEE Transactions on Industrial Electronics 61 (2014) 4912–4919. doi:10.1109/tie.2013.2289867.

[11] M. Hamdi, D. Mulvaney, Prioritised A* search in real-time elevator dispatching, Control Engineering Practice 15 (2007) 219–230. doi:10.1016/j.conengprac.2006.06.005.

[12] B. Bolat, P. Cortés, Genetic and tabu search approaches for optimizing the hall call—Car allocation problem in elevator group systems, Applied Soft Computing 11 (2011) 1792–1800. doi:10.1016/j.asoc.2010.05.023.

[13] P. Cortés, J. Larrañeta, L. Onieva, Genetic algorithm for controllers in elevator groups: analysis and simulation during lunchpeak traffic, Applied Soft Computing 4 (2004) 159–174. doi:10.1016/j.asoc.2003.11.002.

[14] R. H. Crites, A. G. Barto, Elevator Group Control Using Multiple Reinforcement Learning Agents, Machine Learning 33 (1998) 235–262. doi:10.1023/a:1007518724497.

[15] P. E. Utgoff, M. E. Connell, Real-Time Combinatorial Optimization for Elevator Group Dispatching, IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans 42 (2012) 130–146. doi:10.1109/tsmca.2011.2157134.

[16] P. Cortés, J. Muñuzuri, A. Vázquez-Ledesma, L. Onieva, Double deck elevator group control systems using evolutionary algorithms: Interfloor and lunchpeak traffic analysis, Computers & Industrial Engineering 155 (2021) 107190. doi:10.1016/j.cie.2021.107190.

[17] Y. Wu, J. Yang, Directional optimization of elevator scheduling algorithms in complex traffic patterns, Applied Soft Computing 158 (2024) 111567. doi:10.1016/j.asoc.2024.111567.

[18] A. Gharbi, Exploring Heuristic and Optimization Approaches for Elevator Group Control Systems, Applied Sciences 14 (2024) 995. doi:10.3390/app14030995.

[19] J. Sorsa, H. Ehtamo, J.-M. Kuusinen, M. Ruokokoski, M.-L. Siikonen, Modeling uncertain passenger arrivals in the elevator dispatching problem with destination control, Optimization Letters 12 (2018) 171–185. doi:10.1007/s11590-017-1130-0.

[20] M.-L. Siikonen, A. T. So, Verification of equations for up-peak traffic analysis of elevator destination control system based on calculation and simulation, Building Services Engineering Research and Technology (2026). doi:10.1177/01436244261469054.

[21] T. Yamauchi, R. Ide, T. Sugawara, Fair and Effective Elevator Car Dispatching Method in Elevator Group Control System using Cameras, Procedia Computer Science 159 (2019) 455–464. doi:10.1016/j.procs.2019.09.200.

[22] Z. Zhang, Y. Zheng, H. Xu, H. Li, A novel elevator group control algorithm based on binocular-cameras corridor passenger detection and tracking, Multimedia Tools and Applications 74 (2015) 1761–1775. doi:10.1007/s11042-013-1716-1.

[23] J.-H. Kim, B.-R. Moon, Adaptive elevator group control with cameras, IEEE Transactions on Industrial Electronics 48 (2001) 377–382. doi:10.1109/41.915416.

[24] H.-C. Shih, A robust occupancy detection and tracking algorithm for the automatic monitoring and commissioning of a building, Energy and Buildings 77 (2014) 270–280. doi:10.1016/j.enbuild.2014.03.069.

[25] D. Li, C. C. Menassa, V. R. Kamat, Robust non-intrusive interpretation of occupant thermal comfort in built environments with low-cost networked thermal cameras, Applied Energy 251 (2019) 113336. doi:10.1016/j.apenergy.2019.113336.

[26] L.-H. Yang, J. Liu, F.-F. Ye, Y.-M. Wang, C. Nugent, H. Wang, L. Martínez, Highly explainable cumulative belief rule-based system with effective rule-base modeling and inference scheme, Knowledge-Based Systems 240 (2022) 107805. doi:10.1016/j.knosys.2021.107805.

[27] H. Xiong, H. Wang, S. Shi, K. Chen, Comparison study of dispatching rules and heuristics for online scheduling of single machine scheduling problem with predicted release time jobs, Expert Systems with Applications 243 (2024) 122752. doi:10.1016/j.eswa.2023.122752.

[28] H. Mao, J. Yuan, The performance of priority rules for the decentralized resource-constrained multi-project scheduling, Knowledge-Based Systems 304 (2024) 112530. doi:10.1016/j.knosys.2024.112530.

[29] J. Luo, M. Vanhoucke, J. Coelho, W. Guo, An efficient genetic programming approach to design priority rules for resource-constrained project scheduling problem, Expert Systems with Applications 198 (2022) 116753. doi:10.1016/j.eswa.2022.116753.

[30] C. Barut, G. Yildirim, Y. Tatar, An intelligent and interpretable rule-based metaheuristic approach to task scheduling in cloud systems, Knowledge-Based Systems 284 (2024) 111241. doi:10.1016/j.knosys.2023.111241.

[31] J. Tang, Y. Zheng, An optimization model for rescue team deployment in geological disaster considering priority and fairness, Knowledge-Based Systems 352 (2026) 116912. doi:10.1016/j.knosys.2026.116912.

[32] A. Martínez-Sykora, F. McLeod, T. Cherrett, A. Friday, Exploring fairness in food delivery routing and scheduling problems, Expert Systems with Applications 240 (2024) 122488. doi:10.1016/j.eswa.2023.122488.

[33] A. Ala, V. Simic, D. Pamucar, E. B. Tirkolaee, Appointment Scheduling Problem under Fairness Policy in Healthcare Services: Fuzzy Ant Lion Optimizer, Expert Systems with Applications 207 (2022) 117949. doi:10.1016/j.eswa.2022.117949.

[34] A. So, L. Al-Sharif, R. Chan, Universal elevator round trip time calculation for general traffic conditions with extension to batch arrivals, Building Services Engineering Research and Technology 43 (2022) 539–557. doi:10.1177/01436244221095195.

[35] J. Page, D. Robinson, N. Morel, J.-L. Scartezzini, A generalised stochastic model for the simulation of occupant presence, Energy and Buildings 40 (2008) 83–98. doi:10.1016/j.enbuild.2007.01.018.

[36] S. C. Chu, C. Lin, S. Lam, Hospital lift system simulator: A performance evaluator–predictor, European Journal of Operational Research 146 (2003) 156–180. doi:10.1016/s0377-2217(02)00203-5.

[37] M. Jung, M. Park, H.-S. Lee, S. Chi, Agent-Based Lift System Simulation Model for High-Rise Building Construction Projects, Journal of Computing in Civil Engineering 31 (2017) 04017064. doi:10.1061/(asce)cp.1943-5487.0000707.

[38] T. Miyamoto, S. Yamaguchi, MceSim: A Multi-Car Elevator Simulator, IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences E91-A (2008) 3207–3214. doi:10.1093/ietfec/e91-a.11.3207.

[39] E. O. Tartan, C. Çiflikli, Esra (elevator simulation, research & analysis): an open-source software tool for elevator traffic simulation, research, and analysis, Journal of Simulation 18 (2024) 868–885. doi:10.1080/17477778.2024.2330432.

[40] M. Shiomi, M. Kakio, T. Miyashita, How long is too long? Examining waiting times and stress in human-elevator interaction, PeerJ 13 (2025) e19913. doi:10.7717/peerj.19913.

[41] P. Royston, M. K. B. Parmar, Restricted mean survival time: an alternative to the hazard ratio for the design and analysis of randomized trials with a time-to-event outcome, BMC Medical Research Methodology 13 (2013) 152. doi:10.1186/1471-2288-13-152.

[42] Anonymised for review, [dataset] Simulation code, data and passenger-event records (title withheld for anonymised review), Persistent repository record; identifier given on the title page (2026).

## Appendix A. Historical factorial campaign

Table A.1 lists the factors of the historical campaign summarised in Section 4.4. The historical simulator is the legacy model: it predates the corrections of Section 4.1 and uses no door or transfer delay. Its exposure, noise and guidance outputs are not outcomes.

Table A.1: Historical factorial design.

| Factor | Levels |
|---|---|
| Floors | 8, 16, 32 |
| Cars | 2, 4, 8 |
| Population scale | 200, 800, 2,000 |
| Traffic | Up-peak, down-peak, lunch, mixed |
| Priority-request share | 0.02, 0.08, 0.15 |
| Carrying-object share | 0.05, 0.20, 0.40 |
| Synthetic noise | 0.00, 0.08, 0.18 |
| Configurations and seeds | Eight historical configurations; seeds 0–11 |

*Bounds on all-arrival waiting.* Without individual observation times, the historical CSV bounds the finite-horizon waiting burden only broadly. For the layered configuration, the completed-trip contribution per generated passenger is 190.43 units, and adding the maximum possible accrued wait for unresolved passengers gives an upper bound of 1,339.76; the corresponding range for the reference is 203.92 to 1,284.97. The ranges overlap, so the data cannot determine which configuration has lower all-arrival waiting.

*Fairness summaries.* The archived fairness_disparity field is an average of run-level priority/general waiting ratios with a denominator floor of one, not the ratio of the two aggregate group means, and low general-group waits make it unstable. The two summaries point in opposite directions: the archived mean of run-level ratios rises from 1.00 to 2.48 for the layered configuration, while the ratio of aggregate priority to general means falls from 0.95 to 0.36. The difference arises from the aggregation operations and small denominators, so neither should be read as a measure of improved or worsened fairness, and neither ratio alone defines an appropriate allocation of service. The main text therefore reports each group's wait directly.

## Appendix B. Experiment history

Table B.1 distinguishes completed executions from merged tables and reanalyses. Historical stages overlap in scenarios and seeds; they are not independent confirmations. Filtering, hardware, sensing and interaction experiments were not run and are not listed.

Table B.1: Retained execution and analysis history.

| Stage | Runs or rows | Interpretation |
|---|---|---|
| Initial smoke | 12 | Pipeline execution |
| Second smoke | 12 | Repeated pipeline execution |
| Pilot | 7,680 | Six retained configurations |
| Reduced main | 122,472 | Seven configurations, six seeds |
| Layered follow-up | 17,496 | One added configuration, six seeds |
| Combined reduced table | 139,968 | Merge only; no new execution |
| Historical factorial campaign | 279,936 | Eight configurations, twelve seeds |
| Historical validation | 279,936 reused | Descriptive reanalysis |
| Packaging smoke | 12 | Execution check |
| Archived-case spot checks | 8 | Selected case replay |
| Event-recording smoke | 4 | Instrumentation check |
| Legacy event-recording campaign | 5,184 | Superseded; Appendix C |
| Event-file recovery | 1 replay | Corrupt record recovery; aggregates unchanged |
| Corrected-model smoke | 26 | Development check; not part of the main grid |
| Corrected matched-arrival campaign | 16,848 | Thirteen configurations; 22,998,183 passenger records |
| Horizon reanalysis | 7,776 reused | Six corrected configurations; no new simulation |
| Replay parity, Python 3.9.6 | 3,888 replayed | All match the historical archive |
| Replay parity, Python 3.11.15 | 3,888 replayed | 3,105 match the historical archive |

Controller means, group outcomes, scenario intervals, horizon sensitivity, censored-wait sensitivity and the replay-parity records are deposited with the versioned releases [42]. The historical source snapshot was collected after the run; Appendix C shows that it reproduces the archived outputs.

## Appendix C. Legacy event-recording campaign

Before the corrected model was fixed, a 5,184-run campaign recorded passenger events with the legacy simulator for the same 108 scenarios and twelve seeds as Section 4.1. It compared the collective reference, the occupancy rule, the priority rule and the distance-only nearest car, retained 7,076,364 passenger records, and ran on a local workstation with eight Python worker processes (Python 3.12.14). It is superseded for three reasons: the legacy simulator retains stale targets, it draws tie-breaks from configuration-specific random streams (Section 4.1), and its priority rule adds the occupancy and priority terms together relative to the collective reference, so the contrast does not isolate priority.

Tables C.1 and C.2 summarise the results. The direction of the service-allocation effect matches the corrected model: priority requests wait less and general requests wait longer. The magnitudes differ. Relative to collective dispatch, the general-request cost is 23.43 units here and 4.66 units in the corrected model without delays, and the overall difference is +7.47 ($[-2.87, 19.24]$) here and −8.73 ($[-12.01, -5.73]$) in the corrected model. The legacy values are therefore not used as evidence in the main text.

Table C.1: Legacy-model restricted waits (900 model time units) over all generated passengers and mean run-level completion.

| Configuration | All | Priority | General | Completion (%) |
|---|---|---|---|---|
| Priority rule | 294.23 | 99.06 | 310.48 | 82.31 |
| Collective reference | 286.77 | 283.38 | 287.05 | 84.04 |
| Distance-only nearest car | 303.32 | 299.84 | 303.65 | 83.77 |
| Occupancy rule | 297.79 | 294.06 | 298.09 | 82.66 |

Table C.2: Legacy-model paired priority-rule contrasts; positive waiting differences indicate longer waits under the priority rule.

| Reference | Outcome | Difference | 95% interval |
|---|---|---|---|
| Collective reference | All restricted wait | 7.47 | $[-2.87, 19.24]$ |
| Collective reference | Priority restricted wait | $-184.33$ | $[-226.90, -142.33]$ |
| Collective reference | General restricted wait | 23.43 | $[12.26, 35.29]$ |
| Collective reference | Completion (percentage points) | $-1.73$ | $[-3.06, -0.61]$ |
| Distance-only nearest car | All restricted wait | $-9.09$ | $[-26.38, 8.33]$ |
| Distance-only nearest car | Priority restricted wait | $-200.78$ | $[-242.45, -158.27]$ |
| Distance-only nearest car | General restricted wait | 6.83 | $[-11.41, 24.65]$ |
| Distance-only nearest car | Completion (percentage points) | $-1.46$ | $[-3.23, 0.19]$ |

*Replay parity.* The 3,888 legacy-configuration runs of this campaign were also compared with their counterparts in the 279,936-run historical archive. Under Python 3.12.14, 3,105 matched every archived metric to within $10^{-12}$ and 783 did not. Arrival counts matched in every case, so the differences arise in service dynamics rather than demand. They were concentrated in congested buildings: 642 of 1,296 runs (49.5%) with eight floors and two cars differed, against 129 (10.0%) with sixteen floors and four cars and 12 (0.9%) with thirty-two floors and eight cars. The rate did not depend on configuration (18.8% to 22.0%), priority share or seed, and the median absolute difference in mean wait among differing runs was 8.95 units.

The cause is the movement rule of Section 3.3. A stationary car chooses its nearest target with Python's `min` over a set of floors, so equidistant targets are resolved by set iteration order, which differs between Python 3.9 and Python 3.11 or later. Equidistant targets are frequent when few cars serve few floors. Replaying the archived post-run code snapshot under Python 3.9.6 reproduced all 3,888 runs on all eighteen archived metrics to within $10^{-12}$, while Python 3.11.15 reproduced the same 3,105 runs as Python 3.12.14. The historical archive is therefore reproducible from its snapshot under Python 3.9. The corrected model shares the movement rule and reproduces under Python 3.11 or later: 30 of 30 sampled eight-floor runs matched the archived outputs under Python 3.11.15, against 13 of 30 under Python 3.9.6. One incomplete legacy event file was recovered by replaying its fixed case, and every aggregate metric matched its original row.